MEDS · MAKTAB-E-DIGITAL SYSTEMS

MEDS-S1
Project Catalogue

31 projects

UET Lahore · Department of Electrical Engineering · August 2026

Every project on the following pages is a real component of MEDS-S1, with a specification, a testbench and a merge gate. There is no practice work in this catalogue. Read the page for your assigned project carefully, and talk to your mentor about anything that is unclear before you start.

16	8	7
Part One projects 1–1.5 months	Part Two projects 2–3 months	Part Three projects ~3 months

MEDS-S1 is an open RISC-V SoC platform whose purpose is to be attached to. Apache-2.0.
Companion documents: specs/MEDS-S1-SPECIFICATION.md · specs/INTERFACES.md · specs/SCOPE_CONTRACT.md · EXECUTION_PLAN.md

How to use this catalogue

Read this page before you read the projects

> **Read the page for your assigned project.** Priority markings say which ones start now: **P0** begins immediately and is on the critical path, **P1** starts this cycle, **P2** is queued for the next one.

WHAT EVERY PAGE TELLS YOU

- **Duration** — how long the project runs. Durations assume part-time work alongside coursework.
- **Literature review** — every project starts with reading and produces a written memo or note before implementation begins. This is not a warm-up exercise; the note is a deliverable and it is reviewed. Design review before RTL, always.
- **Definition of done** — binary criteria. A project is not finished because the code exists.

THE CONTRIBUTION LADDER

- **T0 — Contributor.** Add a peripheral via soc.yaml, write a driver, write a directed testbench, add a benchmark. Entry: rv-workshop completed.
- **T1 — Implementer.** Implement a module against a frozen spec. Entry: two merged T0 PRs.
- **T2 — Owner.** Own a module and its spec; review T1 work. Entry: one T1 module delivered *and verified*.
- **T3 — Architect.** Own an interface. Only these people may change INTERFACES.md.

Everyone starts at T0, including people who think they should not. The tiers are a ladder, not a label — someone who delivers a smaller project well is a stronger candidate for a larger one next cycle than someone who was handed one this time.

> **Two rules that are not negotiable.** Design review before RTL — a module owner presents the spec and the testbench plan, and the review approves the *spec*, not the code. And blockers are raised the same day: someone blocked for a week has lost a sixth of a semester.

Project Index — Part One

16 projects · Training completion projects — small, bounded, 1 to 1.5 months

ID	PROJECT	TRACK	DURATION	PRIORITY	WP
M-01	UART Peripheral Integration and HAL Driver	Fabric & Periph	6 weeks	P0	WP9, WP14
M-02	SPI Master Peripheral, Driver and Flash Boot Path	Fabric & Periph	6 weeks	P1	WP9, WP14
M-03	GPIO and Timer Peripherals with Drivers	Fabric & Periph	5 weeks	P1	WP9, WP14
M-04	Boot ROM and Second-Stage Loader	Software & BSP	6 weeks	P1	WP9, WP14
M-05	PMA Check Unit — Directed Testbench	Verification	5 weeks	P1	WP13
M-06	ALU and Forwarding Network — Unit Testbench and Coverage	Verification	5 weeks	P1	WP13
M-07	CoreMark and Dhrystone Benchmark Harness	Software & BSP	5 weeks	P1	WP19
M-08	Embench-IoT Port and Automated Regression	Software & BSP	6 weeks	P2	WP19
M-09	libs1_perf — the Performance Measurement Library	Software & BSP	6 weeks	P0	WP14, WP19
M-10	Lint, Coding Standard and CI Documentation Gates	Docs & Research	4 weeks	P0	WP1, WP22
M-11	RISC-V Architectural Compatibility Tests (ACT) — Triage and Reporting	Verification	6 weeks	P0	WP12
M-12	Xilinx IP Survey and Evaluation for KC705 Synthesis	FPGA & Tools	6 weeks	P0	WP16
M-13	Module Documentation and Interface-Contract Sweep	Docs & Research	4 weeks	P1	WP22
M-14	Literature Review — Open-Source RISC-V Core Microarchitectures	Docs & Research	4 weeks	P0	WP0
M-15	Literature Review — Accelerator Coupling and Measurement Methodology	Docs & Research	5 weeks	P1	WP0, WP17
M-16	The Bridge Exercise — Port a Workshop Core into the MEDS-S1 Harness	Verification	6 weeks	P1	WP13

Project Index — Part Two

8 projects · Larger components — one owner, one spec, one testbench

ID	PROJECT	TRACK	DURATION	PRIORITY	WP
T-01	Core Frontend — PC, Branch Prediction and Fetch Interface	Core	10 weeks	P0	WP2
T-02	Core Backend — Decode, Register File, ALU and Hazard Control	Core	12 weeks	P0	WP3
T-03	CSR File, Trap Architecture and Performance Counters	Core	12 weeks	P0	WP4
T-04	AXI4 Fabric — Crossbar, Width Adaptation and Address Decode	Fabric & Periph	10 weeks	P0	WP8
T-05	CLINT and PLIC — the Interrupt Subsystem	Fabric & Periph	8 weeks	P1	WP9
T-06	Board Support Package — crt0, newlib, HAL and make run	Software & BSP	10 weeks	P0	WP14
T-07	RISCOF, Architectural Tests and Co-simulation in CI	Verification	8 weeks	P0	WP12
T-08	KC705 Board Port — DDR3 Bring-Up and Timing Closure	FPGA & Tools	12 weeks	P1	WP16

Project Index — Part Three

7 projects · Extra-large components — on the critical path, or an interface

ID	PROJECT	TRACK	DURATION	PRIORITY	WP
R-01	**Completion Buffer, Retire Logic and the MXIF Port**	Core	12 weeks	P0 — critical path	WP5
R-02	**Load/Store Unit, Store Buffer, PMP and Atomics**	Memory	12 weeks	P0	WP6
R-03	**Instruction and Data Caches, Zicbom and the SRAM Wrapper**	Memory	12 weeks	P1	WP7
R-04	**The SoC Generator — soc.yaml to RTL, Linker, Headers and DTS**	FPGA & Tools	12 weeks	P0	WP10
R-05	**RVFI Trace Port and the Spike Co-simulation Harness**	Verification	10 weeks	P0	WP11
R-06	**Debug Module, JTAG DTM, OpenOCD and Semihosting**	FPGA & Tools	10 weeks	P0	WP15
R-07	**Accelerator Socket and the Conformance Testbenches**	Memory	10 weeks	P1	WP17

UART Peripheral Integration and HAL Driver

Fabric & Periph

DURATION	PRIORITY	PREREQUISITES
6 weeks	**P0**	rv-workshop; basic SystemVerilog; C

Bring a reused UART onto the AXI4-Lite peripheral subtree, give it an entry in soc.yaml, and write the HAL driver and printf backend the whole lab will use for the next four years. This is the smallest complete slice of the platform: an IP, a bus wrapper, a testbench, a driver and a C program that prints.

WHAT YOU WILL BUILD

- Evaluate the OpenTitan and PULP UART IP; choose one and record the licence
- AXI4-Lite wrapper plus a soc.yaml entry — base address, IRQ line, PMA attributes
- Directed testbench: TX, RX, framing error, FIFO full and empty, IRQ assert and clear
- HAL driver: uart_init, uart_putc, uart_getc, uart_write
- newlib _write hook so printf() works over UART

LITERATURE REVIEW & THE NOTE IT PRODUCES

- OpenTitan UART and PULP APB UART specifications
- INTERFACES.md P1–P5 — PMA rules for device regions
- Deliverable: a 2-page memo comparing the two IPs on features, FIFO depth, IRQ model, licence and RTL size, ending in a recommendation

DELIVERABLES

- RTL wrapper in rtl/peripherals/ and the soc.yaml entry
- Unit testbench running in CI
- HAL driver and header in sw/bsp/
- IP comparison memo, and a module README stating the interface contract

DEFINITION OF DONE

- printf("Hello World\n") from C runs on the Verilator board
- Testbench green in CI; Verible lint clean with no waivers
- README.md present in the module directory (NFR-7)

MILESTONES

W1	Literature review and IP selection memo
W2–3	AXI4-Lite wrapper and soc.yaml entry
W4	Directed testbench
W5	HAL driver and printf backend
W6	Integration, documentation, PR review

Start here: Spec §24 (peripherals), SCOPE_CONTRACT §5 (reuse policy), EXECUTION_PLAN WP9/WP14

NOTES · QUESTIONS FOR YOUR MENTOR

SPI Master Peripheral, Driver and Flash Boot Path

Fabric & Periph

DURATION	PRIORITY	PREREQUISITES
6 weeks	**P1**	rv-workshop; SystemVerilog; a little C

Give MEDS-S1 an SPI master so it can reach QSPI flash and SD cards. This is the path by which a second-stage bootloader and a multi-megabyte weight file get onto the board without a debugger attached — which matters the moment an accelerator needs real model data.

WHAT YOU WILL BUILD

- Select and wrap an SPI master IP; AXI4-Lite register window, soc.yaml entry
- Support modes 0 and 3, configurable clock divider, and a chip-select bank
- Directed testbench against a behavioural SPI flash model
- HAL driver: spi_init, spi_xfer, spi_read_flash
- A worked example reading a known pattern out of the flash model

LITERATURE REVIEW & THE NOTE IT PRODUCES

- The JEDEC SFDP basics and one QSPI flash datasheet (e.g. Micron N25Q)
- SD card SPI-mode initialisation sequence
- Deliverable: a 1-page note on what the second-stage loader will need from this driver, agreed with the M-04 team before you build

DELIVERABLES

- SPI RTL wrapper, soc.yaml entry, unit testbench in CI
- HAL driver plus the flash-read example application
- Module README with the register map and the interface contract

DEFINITION OF DONE

- Testbench reads a known pattern from the behavioural flash model, in CI
- Driver API agreed and used by the M-04 boot-loader project
- Lint clean; README present

MILESTONES

W1	Reading, IP selection, interface note agreed with M-04
W2–3	RTL wrapper and register window
W4	Testbench and flash model
W5	HAL driver
W6	Example application and documentation

Start here: Spec §24, FR-15 (boot from ROM, second stage from UART/SD/QSPI), EXECUTION_PLAN WP9

NOTES · QUESTIONS FOR YOUR MENTOR

GPIO and Timer Peripherals with Drivers

Fabric & Periph

DURATION	PRIORITY	PREREQUISITES
5 weeks	**P1**	rv-workshop; SystemVerilog basics

Deliver the two peripherals every board bring-up depends on. GPIO is how the first LED blinks and how bring-up step 1 is passed; the timer is how software measures anything before the performance counters are trusted. Small, well-defined, and on the critical path of the first FPGA milestone.

WHAT YOU WILL BUILD

- GPIO: N-bit configurable direction, output, input synchroniser, per-pin interrupt
- Timer: free-running counter, compare registers, periodic and one-shot interrupt
- Both with AXI4-Lite register windows and soc.yaml entries
- Directed testbenches for both, including the interrupt paths
- HAL drivers and a blink plus periodic-interrupt example

LITERATURE REVIEW & THE NOTE IT PRODUCES

- The CLINT mtime/mtimecmp model in the RISC-V privileged specification
- Read spec §24 and note how a peripheral IRQ reaches the core through the PLIC
- Deliverable: a half-page note on why the timer here is not the same thing as CLINT mtime, and when software should use each

DELIVERABLES

- Two RTL peripherals, two soc.yaml entries, two unit testbenches in CI
- Two HAL drivers plus the example application
- Module READMEs for both

DEFINITION OF DONE

- LED blinks on the Verilator board from C, driven by the timer interrupt
- Both testbenches green in CI; lint clean
- Interrupt paths exercised in simulation, not just the register reads

MILESTONES

W1	Reading and register-map design, reviewed before RTL
W2	GPIO RTL and testbench
W3	Timer RTL and testbench
W4	HAL drivers
W5	Example application and documentation

Start here: Spec §24, RISC-V Privileged Spec (CLINT), EXECUTION_PLAN WP9

NOTES · QUESTIONS FOR YOUR MENTOR

Boot ROM and Second-Stage Loader

Software & BSP

DURATION	PRIORITY	PREREQUISITES
6 weeks	**P1**	C, assembly, linker scripts; rv-workshop

Write the first code that runs on MEDS-S1. The boot ROM sets up the machine from reset and hands over to a second-stage loader that can pull an image in over UART or QSPI. Everything anybody ever runs on this platform passes through the few hundred instructions you write here.

WHAT YOU WILL BUILD

- Reset vector, stack setup, BSS clear, trap-vector install, jump to second stage
- A 32 KB ROM image generated into the RTL from the build, never hand-edited
- Second-stage loader: XMODEM-style receive over UART, plus a QSPI path using M-02's driver
- Platform version reporting — read MEDS_S1_PLATFORM_VERSION and print an identifying banner
- A boot-time self-test that checks SRAM is readable and writable before handing over

LITERATURE REVIEW & THE NOTE IT PRODUCES

- Read two existing boot ROMs — the CVA6 and the Ibex/OpenTitan ones — and note what each does before jumping to user code
- The generated linker script and memory map in soc.yaml
- Deliverable: a 1-page boot-flow diagram, from reset vector to main(), reviewed before any code is written

DELIVERABLES

- crt0-adjacent boot ROM source in sw/boot/, plus the ROM image generation step
- Second-stage loader with UART and QSPI paths
- Boot-flow diagram and a README that documents every ROM entry point

DEFINITION OF DONE

- A cold reset in Verilator reaches main() in a C program and prints the version banner
- The loader receives a small ELF over the UART model and runs it
- ROM image is build-generated; no checked-in binary blob

MILESTONES

W1	Reading and the boot-flow diagram
W2–3	Boot ROM and ROM generation
W4	UART second-stage path
W5	QSPI path with M-02
W6	Self-test, banner, documentation

Start here: Spec §24, FR-15, SCOPE_CONTRACT §2.2, EXECUTION_PLAN WP9

NOTES · QUESTIONS FOR YOUR MENTOR

PMA Check Unit — Directed Testbench

Verification

DURATION	PRIORITY	PREREQUISITES
5 weeks	**P1**	SystemVerilog; willingness to read a spec closely

Write the testbench that proves the PMA check unit enforces every region attribute correctly. This is the module that decides whether a load is cacheable, whether a device access may be reordered, and whether an access to a hole traps — and the failure mode when it is wrong is a bug that looks like a cache bug for three weeks.

WHAT YOU WILL BUILD

- A testbench covering all six attributes across every region in the default memory map
- Cross coverage: region × access type × size × alignment
- Negative tests: access to an unmapped hole, misaligned atomic, write to a read-only region
- Ordering checks for order:strong device regions (INTERFACES.md P2)
- A self-checking scoreboard, not eyeballed waveforms

LITERATURE REVIEW & THE NOTE IT PRODUCES

- Spec §11 (PMP and PMA checking) and the PMA table, in full
- RISC-V Privileged Spec, the physical memory attributes chapter
- Deliverable: a 1-page table of every region and the attribute set you will test, signed off by the WP6 owner before you write the testbench

DELIVERABLES

- verif/unit/tb_pma_check.sv with a self-checking scoreboard
- A functional coverage group for the PMA cross
- A short report of the coverage achieved and any holes you could not close, with reasons

DEFINITION OF DONE

- Testbench runs in CI on every PR and takes under 60 seconds
- Every region in the memory map is exercised for every attribute
- Coverage report published; any uncovered bin has a written reason

MILESTONES

W1	Reading and the region/attribute table
W2	Testbench skeleton and stimulus
W3	Scoreboard and negative tests
W4	Coverage model
W5	CI integration and the coverage report

Start here: Spec §11, INTERFACES.md P1–P5, EXECUTION_PLAN WP13

NOTES · QUESTIONS FOR YOUR MENTOR

ALU and Forwarding Network — Unit Testbench and Coverage

Verification

DURATION	PRIORITY	PREREQUISITES
5 weeks	**P1**	SystemVerilog; digital design fundamentals

Prove the arithmetic and the forwarding paths are right, exhaustively where you can and by constrained-random where you cannot. Forwarding bugs are the classic pipeline defect: they pass every simple test and fail on a specific back-to-back sequence. A good testbench here saves the WP3 owner weeks.

WHAT YOU WILL BUILD

- Exhaustive testing of the ALU operations that admit it, and directed corner cases for the rest
- A constrained-random layer generating back-to-back dependent instruction pairs and triples
- Reference model in SystemVerilog or C for result checking
- Coverage of the full EX->EX and MEM->EX forwarding matrix from spec §8.1
- The load-use stall case explicitly, since it is the one hazard that is not forwarded

LITERATURE REVIEW & THE NOTE IT PRODUCES

- Spec §8 (hazards, forwarding and stalls) including the hazard table, in full
- One chapter of a standard architecture text on data hazards, for the vocabulary
- Deliverable: a filled-in copy of the §8.2 hazard table marking which row each of your tests covers

DELIVERABLES

- verif/unit/tb_alu.sv and verif/unit/tb_forwarding.sv
- Reference model and the coverage groups
- The annotated hazard table showing test-to-hazard traceability

DEFINITION OF DONE

- Every row of the §8.2 hazard table has at least one directed test
- Forwarding matrix coverage at 100%, report published
- Both testbenches in CI and green

MILESTONES

W1	Reading and the hazard-table traceability map
W2	ALU testbench and reference model
W3	Forwarding testbench
W4	Constrained-random layer
W5	Coverage closure and CI integration

Start here: Spec §7, §8, EXECUTION_PLAN WP13

NOTES · QUESTIONS FOR YOUR MENTOR

CoreMark and Dhrystone Benchmark Harness

Software & BSP

DURATION	PRIORITY	PREREQUISITES
5 weeks	**P1**	C, make, shell scripting

Stand up the two benchmarks the outside world compares cores on, and automate them so a number appears on every pull request without anybody asking. NFR-2 sets a target of 1.0 CoreMark/MHz; you build the thing that says whether we met it, and that notices the day we stop meeting it.

WHAT YOU WILL BUILD

- Port CoreMark and Dhrystone to the BSP, with correct timing via mcycle
- A results parser producing machine-readable output, not just console text
- CI integration reporting CoreMark/MHz and DMIPS/MHz on each PR
- A regression gate: a drop of more than 3% blocks the merge (NFR-10)
- A results-history file so the numbers can be plotted over the life of the project

LITERATURE REVIEW & THE NOTE IT PRODUCES

- The CoreMark run rules — particularly what may and may not be changed, and which compiler flags are reportable
- Why Dhrystone is criticised, and what it is still useful for
- Deliverable: a 1-page note stating the exact flags, library and reporting form we will use, so our numbers stay comparable to published ones

DELIVERABLES

- sw/benchmarks/ with both benchmarks and the run scripts
- CI job publishing the results table
- The run-rules note and a README on reproducing a number by hand

DEFINITION OF DONE

- make bench BOARD=verilator prints a CoreMark/MHz figure
- CI publishes the number on every PR and blocks on a >3% regression
- The reported configuration and flags are documented alongside every number

MILESTONES

W1	Run rules note and toolchain flags
W2–3	Porting both benchmarks onto the BSP
W4	Parser and CI job
W5	Regression gate and history file

Start here: Spec §32.1, NFR-2, NFR-10, EXECUTION_PLAN WP19

NOTES · QUESTIONS FOR YOUR MENTOR

Embench-IoT Port and Automated Regression

Software & BSP

DURATION	PRIORITY	PREREQUISITES
6 weeks	**P2**	C, make, Python; M-07 running first is helpful

Embench-IoT is the modern, harder-to-game embedded benchmark suite, and it is the one a reviewer will ask about. Porting all of it gives per-benchmark cycle counts that show where the core is weak — which is exactly the information a v2 microarchitecture project needs in order to be worth doing.

WHAT YOU WILL BUILD

- Port the Embench-IoT suite to the BSP and the linker script
- Per-benchmark cycle and instruction counts collected through libs1_perf
- A comparison table against the published reference scores
- Nightly CI job, since the full suite is too slow for the per-PR budget
- A short analysis of the three slowest benchmarks and why they are slow

LITERATURE REVIEW & THE NOTE IT PRODUCES

- The Embench-IoT papers and the project's stated design goals
- Spec §32.1 and §34 (comparison methodology and the required reporting table)
- Deliverable: a 2-page memo on what Embench measures that CoreMark does not, with the specific benchmarks that expose each difference

DELIVERABLES

- sw/benchmarks/embench/ with the full ported suite
- Nightly CI job and the results table in the evidence bundle format
- The analysis memo on the three slowest benchmarks

DEFINITION OF DONE

- Every benchmark in the suite builds and runs to a correct result
- Nightly job publishes a per-benchmark table
- The analysis identifies at least one concrete, measurable v2 improvement

MILESTONES

W1	Reading and the memo
W2–4	Porting, benchmark by benchmark
W5	CI job and results table
W6	Analysis of the slow cases

Start here: Spec §32.1, §34, EXECUTION_PLAN WP19

NOTES · QUESTIONS FOR YOUR MENTOR

libs1_perf — the Performance Measurement Library

Software & BSP

DURATION	PRIORITY	PREREQUISITES
6 weeks	**P0**	C; interest in measurement and methodology

Build the library every result from this platform will be measured with. If libs1_perf is good, a 2031 thesis is comparable to a 2027 one; if it is sloppy, none of the numbers this lab publishes can be trusted. Small in code, large in consequence.

WHAT YOU WILL BUILD

- PERF_BEGIN / PERF_END region markers with negligible overhead
- A programmable event-selector API over mhpmcounter3–15 (spec §12)
- Counter save and restore so nested regions work correctly
- A report formatter that emits the §34.1 required reporting table directly
- An overhead self-measurement, so users know the cost of measuring

LITERATURE REVIEW & THE NOTE IT PRODUCES

- Spec §12 (performance counters) and §31 (measurement infrastructure)
- Spec §34 — the required reporting table and the rules around it
- Deliverable: a 1-page note on why kernel-only speedup is misleading and what end-to-end measurement has to include; this becomes the library's doc header

DELIVERABLES

- sw/bsp/libs1_perf/ with the API, headers and the report formatter
- A worked example measuring a small workload end to end
- The methodology note, and a README with the counter event list

DEFINITION OF DONE

- The library emits the §34.1 table for an example workload with no hand editing
- Measurement overhead is itself measured and documented
- Used by at least one benchmark project (M-07 or M-08) before sign-off

MILESTONES

W1	Reading and the methodology note
W2–3	Counter API and region markers
W4	Report formatter for the §34 table
W5	Overhead measurement
W6	Integration with a benchmark and documentation

Start here: Spec §12, §31, §34, EXECUTION_PLAN WP14/WP19

NOTES · QUESTIONS FOR YOUR MENTOR

Lint, Coding Standard and CI Documentation Gates

Docs & Research

DURATION	PRIORITY	PREREQUISITES
4 weeks	**P0**	Some SystemVerilog; care about consistency

Write the rules everyone's RTL is held to, and the CI jobs that enforce them without a human having to nag. This is a small project with unusual leverage: it runs on every commit anybody makes for the next four years, and it is one of the few Phase-0 items that must land before RTL starts.

WHAT YOU WILL BUILD

- CODING_STANDARD.md — naming, file layout, reset style, parameter conventions
- A Verible lint configuration matching it, with a documented waiver process
- A CI job checking every module directory has a README stating its interface contract (NFR-7)
- A CI job checking SPDX headers and licence consistency on every file
- A pre-commit hook so contributors see failures before CI does

LITERATURE REVIEW & THE NOTE IT PRODUCES

- The lowRISC and PULP SystemVerilog style guides
- Appendix D of the specification — RTL naming conventions
- Deliverable: a 1-page summary of where the two style guides disagree and which one MEDS-S1 follows in each case, with the reasoning

DELIVERABLES

- CODING_STANDARD.md, the Verible config, and the waiver process documented
- Two CI jobs (README check, SPDX check) and the pre-commit hook
- The style-guide comparison note

DEFINITION OF DONE

- make ci runs lint and both doc gates and is green on the current tree
- A deliberately malformed test branch is correctly rejected by each gate
- Zero waivers in the tree without a written justification

MILESTONES

W1	Reading and the style comparison note
W2	CODING_STANDARD.md and the Verible config
W3	CI gates
W4	Pre-commit hook, negative testing, documentation

Start here: NFR-4, NFR-7, Appendix D, EXECUTION_PLAN WP1/WP22 and §10 week 3

NOTES · QUESTIONS FOR YOUR MENTOR

RISC-V Architectural Compatibility Tests (ACT) — Triage and Reporting

Verification

DURATION	PRIORITY	PREREQUISITES
6 weeks	**P0**	Linux command line, Python, patience; no RTL experience needed

Run the RISC-V International ACT suite — the architectural compatibility tests, formerly riscv-arch-test — against the reference model and, as it appears, against S1-Core. Triage every failure into a real bug, a test-harness problem or an unimplemented feature. This is mostly process and reporting work rather than design work, and it is the evidence that lets MEDS-S1 claim compatibility at all.

WHAT YOU WILL BUILD

- A working RISCOF setup with Sail as the golden reference model
- The full ACT suite running for RV64IMAC_Zicsr_Zifencei_Zicbom, config by config
- A triage log classifying every failure: core bug, harness problem, or out-of-scope feature
- A compatibility report in the form that goes into the release evidence bundle
- An issue raised, with a minimal reproducer, for every failure classified as a core bug

LITERATURE REVIEW & THE NOTE IT PRODUCES

- The RISCOF documentation and the ACT repository structure and naming rules
- How a test signature is produced and compared, end to end
- Deliverable: a 2-page explainer on what architectural compatibility does and does not prove — this goes to the whole lab, because it is widely misunderstood

DELIVERABLES

- The RISCOF configuration and plugins committed to the repo
- The triage log and the compatibility report
- Issues filed with reproducers; the explainer memo

DEFINITION OF DONE

- The suite runs end to end from a single make target
- Every failure is classified, with a reason, and none are left unexplained
- The report is in evidence-bundle form (spec §28.6)

MILESTONES

W1	Reading and the explainer memo
W2	RISCOF and Sail installed and running
W3–4	Full suite execution and triage
W5	Report and issue filing
W6	Automation so it reruns from one command

Start here: Spec §28.3, §28.6, FR-1, EXECUTION_PLAN WP12; RISCOF and riscv-arch-test upstream

NOTES · QUESTIONS FOR YOUR MENTOR

Xilinx IP Survey and Evaluation for KC705 Synthesis

FPGA & Tools

DURATION	PRIORITY	PREREQUISITES
6 weeks	**P0**	Vivado installed; no RTL design experience needed

Survey the Xilinx IP we will lean on when MEDS-S1 meets real silicon, and produce configuration recipes the FPGA team can use rather than rediscover. Getting MIG and the clocking right is most of what makes an FPGA bring-up take two weeks instead of two months, and none of it requires writing core RTL.

WHAT YOU WILL BUILD

- An evaluation of MIG for DDR3 on KC705: interface width, clocking, calibration, AXI options
- Clocking Wizard, Block Memory Generator and AXI SmartConnect configurations for our topology
- ILA, VIO and JTAG-to-AXI debug cores — what each costs in area and what each is worth
- A small standalone Vivado project per IP that builds and proves the configuration works
- Resource and timing figures for each IP at our target clock, measured, not estimated

LITERATURE REVIEW & THE NOTE IT PRODUCES

- Xilinx PG150 (MIG 7 Series), PG065 (Clocking Wizard), PG058 (BMG), PG247 (SmartConnect)
- The KC705 board user guide, particularly the DDR3 and clock topology
- Deliverable: a 3-page decision memo recommending an IP and a configuration for each function, with the licensing and portability implications stated

DELIVERABLES

- fpga/ip/ with a reproducible Tcl configuration script per IP
- The decision memo and a resource/timing table
- A README describing how to regenerate every IP from scratch

DEFINITION OF DONE

- Each IP configuration builds from its Tcl script with no GUI steps
- Measured resource and f_max numbers recorded for each
- The WP16 owner accepts the memo as the basis for board bring-up

MILESTONES

W1–2	Reading the product guides and the board user guide
W3	MIG evaluation and test project
W4	Clocking, BRAM and interconnect IP
W5	Debug cores
W6	Decision memo and Tcl scripting

Start here: Spec §29 (FPGA implementation), §30 (area/timing budgets), EXECUTION_PLAN WP16

NOTES · QUESTIONS FOR YOUR MENTOR

Module Documentation and Interface-Contract Sweep

Docs & Research

DURATION	PRIORITY	PREREQUISITES
4 weeks	**P1**	Willingness to read other people's RTL and ask questions

NFR-7 requires every module to carry a README stating its interface contract. Making that true — and building the gate that keeps it true — is how this project survives the cohort turnover that C1 says is certain. You will read more of the codebase than almost anyone, which is the real reward here.

WHAT YOU WILL BUILD

- A README template: purpose, ports, parameters, timing assumptions, reset behaviour, owner, backup
- A README for every module that lacks one, written by reading the RTL and interviewing the owner
- A handover-note section, per EXECUTION_PLAN §12, in each module README
- A docs index page linking every module README
- A list of the places where the RTL and the specification disagree, filed as issues

LITERATURE REVIEW & THE NOTE IT PRODUCES

- EXECUTION_PLAN §12 (continuity across cohorts) and §8 (definition of done for a WP)
- Two well-documented open-source hardware repos, for what good looks like
- Deliverable: the README template itself, reviewed and agreed before the sweep starts

DELIVERABLES

- The template, and a README in every module directory
- The docs index and the RTL-versus-spec discrepancy list

DEFINITION OF DONE

- The M-10 CI documentation gate passes across the whole tree
- Every README names an owner and a backup
- Every discrepancy found is filed as an issue, not silently fixed

MILESTONES

W1	Template design and review
W2–3	The sweep, module by module
W4	Index, discrepancy list, issue filing

Start here: NFR-7, EXECUTION_PLAN §8 and §12, spec Appendix D

NOTES · QUESTIONS FOR YOUR MENTOR

Literature Review — Open-Source RISC-V Core Microarchitectures

Docs & Research

DURATION	PRIORITY	PREREQUISITES
4 weeks	**P0**	None. This is the best first project in the catalogue.

Read CVA6, Ibex and Rocket — their structure, not their code — and present what each chose and why. Phase 0 requires this before S1-Core RTL is written, because the cheapest way to avoid a design mistake is to find someone who already made it. Three contributors, one core each, 45 minutes of presentation each.

WHAT YOU WILL BUILD

- One core per person: pipeline structure, hazard strategy, memory interface, privilege support
- A comparison table across all three: stages, issue width, branch prediction, cache structure, extension mechanism, verification approach
- For each core, the single design decision you think is most worth copying, and why
- For each, the decision you think is worth avoiding, and why
- A 45-minute presentation per core, delivered to the lab

LITERATURE REVIEW & THE NOTE IT PRODUCES

- The CVA6, Ibex and Rocket Chip papers and their documentation
- MEDS-S1 SCOPE_CONTRACT §3 — read it after your review and check whether our deferrals still look right to you
- Deliverable: a 6–8 page comparison report, plus three presentations

DELIVERABLES

- The comparison report in docs/reviews/
- Three recorded or delivered presentations
- A short list of proposed changes to the MEDS-S1 design, if you find any

DEFINITION OF DONE

- All three presentations delivered to the lab
- The comparison table is complete with no unfilled cells
- At least one concrete, argued recommendation reaches the Phase-0 design review

MILESTONES

W1	Assign cores; read the papers
W2	Read the documentation and structure; draft sections
W3	Comparison table and report
W4	Presentations

Start here: EXECUTION_PLAN §10 weeks 1–2, SCOPE_CONTRACT §3, ADDENDUM.md

NOTES · QUESTIONS FOR YOUR MENTOR

Literature Review — Accelerator Coupling and Measurement Methodology

Docs & Research

DURATION	PRIORITY	PREREQUISITES
5 weeks	**P1**	None; useful if you are curious about research rather than RTL

Survey how the literature attaches accelerators to processors, and — more usefully — how honestly it measures them. The recurring failure in accelerator papers is a block that was never attached to anything, reporting a kernel-only speedup. Your job is to characterise that failure precisely enough that MEDS-S1's methodology can avoid it.

WHAT YOU WILL BUILD

- A survey of coupling mechanisms: custom instructions, coprocessor interfaces, memory-mapped accelerators, and where each is used in practice
- A read of the CV-X-IF specification against our MXIF profile in INTERFACES.md
- A review of 8–10 recent edge-AI accelerator papers, recording exactly what each measured
- A tally: how many report end-to-end results, how many report kernel-only, how many report energy as measured versus modelled
- A recommendation on what MEDS-S1's §34 reporting table should require

LITERATURE REVIEW & THE NOTE IT PRODUCES

- OpenHW CV-X-IF specification; MEDS-S1 INTERFACES.md §1 and §4
- Spec §21 (choosing a coupling mechanism), §31 and §34
- Deliverable: a 6-page review with the measurement tally as its central table

DELIVERABLES

- The review in docs/reviews/
- A one-page recommendation on the §34 reporting requirements
- A 30-minute presentation to the lab

DEFINITION OF DONE

- At least eight papers reviewed with the measurement tally completed for each
- The recommendation is specific enough to act on, not general advice
- Presented to the lab; the MEDS-V team has read it

MILESTONES

W1	Coupling mechanisms and CV-X-IF
W2–3	Paper survey and the measurement tally
W4	Write-up
W5	Recommendation and presentation

Start here: Spec §19–§21, §31, §34, INTERFACES.md, EXECUTION_PLAN WP0

NOTES · QUESTIONS FOR YOUR MENTOR

The Bridge Exercise — Port a Workshop Core into the MEDS-S1 Harness

Verification

DURATION	PRIORITY	PREREQUISITES
6 weeks	**P1**	rv-workshop completed, with your own single-cycle RV32I core

Build the on-ramp that turns a one-day workshop into a contributor pipeline. You will take a workshop core, wrap it in the platform's memory interface, and run the platform's RV32I architectural tests against it. Most workshop cores fail — informatively. That failure is the best possible demonstration of why the verification harness exists.

WHAT YOU WILL BUILD

- An adapter wrapping a simple workshop core in the platform fetch and memory interfaces
- A cut-down RV32I test flow that runs against it
- A results page showing which tests fail and, for each, what the core got wrong
- A step-by-step guide a new student can follow alone in one afternoon
- The exercise wired into the onboarding path so every new cohort does it

LITERATURE REVIEW & THE NOTE IT PRODUCES

- EXECUTION_PLAN §4.1 (the on-ramp) — this project is that paragraph, built
- The platform fetch_req/fetch_rsp and memory interfaces in INTERFACES.md
- Deliverable: a half-page note predicting which tests you expect a naive core to fail, written before you run anything; compare it with reality afterwards

DELIVERABLES

- The adapter and the cut-down test flow in verif/bridge/
- The student-facing guide and the annotated results page
- Your prediction note and the comparison against actual results

DEFINITION OF DONE

- A student who has done only rv-workshop can complete the exercise in an afternoon, following the guide alone — tested on a real student who is not you
- The failure explanations are specific, not "the test failed"

MILESTONES

W1	Reading and the prediction note
W2–3	Adapter and test flow
W4	Run, collect and explain the failures
W5	Write the guide
W6	Test the guide on a real student and revise

Start here: EXECUTION_PLAN §4.1 and §11, NFR-9, INTERFACES.md

NOTES · QUESTIONS FOR YOUR MENTOR

Core Frontend — PC, Branch Prediction and Fetch Interface

Core

DURATION	PRIORITY	PREREQUISITES
10 weeks	**P0**	Solid SystemVerilog; pipeline fundamentals

Own the front of the pipeline: program counter generation, static BTFN prediction, the fetch_req/fetch_rsp interface and compressed-instruction expansion. The interface is the point — it is specified so the predictor can be swapped later without touching the pipeline, which turns a v2 branch-predictor study into a clean, measurable project with a baseline.

WHAT YOU WILL BUILD

- PC generation with redirect from branch resolution, traps and debug entry
- Static BTFN prediction: backward taken, forward not taken
- The fetch_req/fetch_rsp interface exactly as specified, with the predictor behind it
- C-extension expansion to 32-bit encodings before decode
- Misaligned fetch handling and the instruction-access-fault path
- A unit testbench covering redirect, expansion and every fetch fault case

LITERATURE REVIEW & THE NOTE IT PRODUCES

- Spec §6 and §7.1; INTERFACES.md for the fetch interface contract
- The frontend structure of Ibex and CVA6 — see M-14's review if it has landed
- Deliverable: a 2-page design note on the redirect path and its timing, presented at a design review before RTL is written

DELIVERABLES

- rtl/core/frontend/ with the RTL and its unit testbench
- The design note and a module README stating the interface contract
- A measured baseline branch-prediction accuracy on the benchmark suite

DEFINITION OF DONE

- Design review passed before RTL; testbench green in CI; lint clean
- Fetch interface matches INTERFACES.md with no local amendments
- Baseline prediction accuracy published, so a v2 predictor has something to beat

MILESTONES

W1–2	Reading, design note, design review
W3–5	PC, redirect and fetch interface RTL
W6–7	BTFN and C-expansion
W8–9	Unit testbench and fault cases
W10	Integration, baseline measurement, documentation

Start here: Spec §6, §7.1, INTERFACES.md, SCOPE_CONTRACT §2.1, EXECUTION_PLAN WP2

NOTES · QUESTIONS FOR YOUR MENTOR

Core Backend — Decode, Register File, ALU and Hazard Control

Core

DURATION	PRIORITY	PREREQUISITES
12 weeks	**P0**	Strong SystemVerilog; comfortable with the ISA specification

Own the middle of the pipeline — decode, the register file, the ALU, the full forwarding network and the hazard control that ties them together. This is the largest core package and everything downstream depends on it, including the completion buffer that sits on the project's critical path.

WHAT YOU WILL BUILD

- Decoder for RV64IMAC_Zicsr_Zifencei, generated from a machine-readable instruction list
- Register file with the required read ports and write arbitration
- ALU, branch resolution and the address-generation unit
- Full EX->EX and MEM->EX forwarding per spec §8.1, and single-cycle load-use stall
- The hazard unit implementing every row of the §8.2 hazard table
- The multi-cycle unit dispatch port that MUL, DIV and MXIF all attach to

LITERATURE REVIEW & THE NOTE IT PRODUCES

- Spec §7 and §8 in full; the RISC-V unprivileged specification for the instruction set
- Spec §9 — read it early, because the completion buffer constrains your write-back path
- Deliverable: a 3-page microarchitecture note including the decode table format and the forwarding matrix, presented at a design review before RTL

DELIVERABLES

- rtl/core/backend/ with RTL, the generated decoder and unit testbenches
- The microarchitecture note and module READMEs
- Coordination with M-06, who verifies your ALU and forwarding paths

DEFINITION OF DONE

- Design review passed before RTL
- M-06's forwarding coverage reaches 100% against your implementation
- RV64I architectural tests pass against Sail once the frontend and CSR are present

MILESTONES

W1–2	Reading, microarchitecture note, design review
W3–5	Decoder and register file
W6–8	ALU, branch, AGU
W9–10	Forwarding and hazard unit
W11–12	Multi-cycle dispatch port, integration, documentation

Start here: Spec §7, §8, §9, EXECUTION_PLAN WP3 (critical path)

NOTES · QUESTIONS FOR YOUR MENTOR

CSR File, Trap Architecture and Performance Counters

Core

DURATION	PRIORITY	PREREQUISITES
12 weeks	**P0**	Careful reading of the privileged specification; SystemVerilog

Own privilege. The CSR file, the M-mode trap architecture, the privilege state machine and the performance counters. S-mode CSRs and delegation are architected now even though behaviour is stubbed until Phase 5 — doing that here costs weeks and saves rewriting the trap logic and access-control matrix during Linux bring-up.

WHAT YOU WILL BUILD

- A generated CSR file — the access-control matrix comes from a table, never hand-written
- Full M-mode trap architecture: mstatus, mtvec, mepc, mcause, mtval, mie, mip
- S-mode CSRs, medeleg and mideleg present and architected; translation stubbed
- Privilege FSM including DebugMode, with dcsr, dpc and dscratch
- mcycle, minstret and mhpmcounter3–15 with programmable event selectors and mcountinhibit
- A unit testbench covering every CSR access permission and every trap cause

LITERATURE REVIEW & THE NOTE IT PRODUCES

- The RISC-V privileged specification, machine and supervisor levels, in full
- Spec §10, §12, §13 and Appendix C (the CSR list)
- Deliverable: the complete CSR access-control matrix as a reviewed table, before any RTL — this table is the design

DELIVERABLES

- rtl/core/csr/ with the generator, the RTL and the unit testbench
- The access-control matrix and the counter event list
- Coordination with M-09, who consumes your counters from software

DEFINITION OF DONE

- Every CSR in Appendix C is implemented or explicitly and correctly absent
- Illegal access to every CSR traps correctly, proven by testbench
- Architectural tests for Zicsr pass against Sail

MILESTONES

W1–2	Reading; the access-control matrix; design review
W3–5	CSR file and generator
W6–8	Trap architecture and privilege FSM
W9–10	Performance counters and event selectors
W11–12	Testbench, arch-test bring-up, documentation

Start here: Spec §10, §12, §13, Appendix C, SCOPE_CONTRACT §4, EXECUTION_PLAN WP4

NOTES · QUESTIONS FOR YOUR MENTOR

AXI4 Fabric — Crossbar, Width Adaptation and Address Decode

Fabric & Periph

DURATION	PRIORITY	PREREQUISITES
10 weeks	**P0**	AXI4 protocol knowledge, or willingness to acquire it quickly

Own the interconnect. A 256-bit backbone with 64-bit core ports and a 32-bit AXI4-Lite peripheral subtree, sized against a declared bandwidth budget rather than a guess. Everything in the SoC talks through what you build, and the width choice is what stops MEDS-V being throttled to scalar bandwidth.

WHAT YOU WILL BUILD

- Crossbar configuration using pulp-platform/axi, with our master and slave topology
- Upsizers and downsizers between the 64-bit, 256-bit and 32-bit domains
- Address decode driven from soc.yaml, never hand-maintained
- The AXI4-Lite bridge to the peripheral subtree
- The uncached DRAM alias (spec §18.4) and its PMA handling
- Bandwidth validation against the §18.3 budget, measured in simulation

LITERATURE REVIEW & THE NOTE IT PRODUCES

- The AMBA AXI4 specification — the ordering and outstanding-transaction rules especially
- Spec §18 in full, particularly §18.3 (the bandwidth budget) and §18.4
- Deliverable: a 2-page note validating or challenging the §18.3 bandwidth budget with your own arithmetic, presented before RTL

DELIVERABLES

- rtl/fabric/ with the crossbar configuration, adapters and decode logic
- A fabric testbench with traffic generators at the declared bandwidths
- The bandwidth validation note and module READMEs

DEFINITION OF DONE

- All four named configurations elaborate in CI
- Measured fabric throughput meets the §18.3 budget in simulation
- Address decode is generated; no hand-written duplicate of the memory map exists

MILESTONES

W1–2 AXI reading; bandwidth validation note; design review

W3–5 Crossbar configuration and topology

W6–7 Width adapters

W8 Address decode from soc.yaml

W9–10 Traffic testbench, bandwidth measurement, documentation

Start here: Spec §18, INTERFACES.md, SCOPE_CONTRACT §5, EXECUTION_PLAN WP8

NOTES · QUESTIONS FOR YOUR MENTOR

CLINT and PLIC — the Interrupt Subsystem

Fabric & Periph

DURATION	PRIORITY	PREREQUISITES
8 weeks	**P1**	SystemVerilog; the privileged spec interrupt model

Own how the machine is interrupted. CLINT provides the timer and software interrupts the privileged specification requires; PLIC multiplexes every peripheral and accelerator interrupt into the core. All sources are level-sensitive, which is a deliberate simplification that removes a whole class of lost-interrupt bug.

WHAT YOU WILL BUILD

- CLINT: mtime, mtimecmp per hart, msip, with the correct clock source
- PLIC: priority, pending, enable and threshold registers, claim and complete flow
- Level-sensitive handling for all sources, including accelerator IRQ lines from the sockets
- Interrupt-source allocation driven from soc.yaml
- A testbench covering priority ordering, threshold masking, and the claim/complete race

LITERATURE REVIEW & THE NOTE IT PRODUCES

- The RISC-V PLIC specification and the CLINT model in the privileged specification
- Spec §24 (peripherals and interrupts) and §20.2 (socket IRQ behaviour)
- Deliverable: a 1-page note on the claim/complete race and how your design avoids losing or double-servicing an interrupt

DELIVERABLES

- rtl/peripherals/clint/ and rtl/peripherals/plic/ with unit testbenches
- soc.yaml interrupt allocation schema and generated headers
- The race-condition note and module READMEs

DEFINITION OF DONE

- Priority, threshold and claim/complete all proven by directed testbench
- A timer interrupt reaches a C handler on the Verilator board
- Accelerator socket IRQ lines route correctly through to the core

MILESTONES

W1	Reading and the race note
W2–3	CLINT
W4–6	PLIC
W7	soc.yaml integration
W8	Testbench closure and documentation

Start here: Spec §24, §20.2, RISC-V PLIC spec, EXECUTION_PLAN WP9

NOTES · QUESTIONS FOR YOUR MENTOR

Board Support Package — crt0, newlib, HAL and make run

Software & BSP

DURATION	PRIORITY	PREREQUISITES
10 weeks	**P0**	C, assembly, linker scripts, build systems

Own the software side of the platform. crt0, a working newlib with syscall stubs, malloc that actually works, printf over both UART and semihosting, the per-peripheral HAL, and one make invocation that runs an ELF on any board. NFR-9 says a new contributor gets from clone to running C in under a day, and this package is what makes that true or false.

WHAT YOU WILL BUILD

- crt0.S: reset entry, stack and global pointer setup, BSS clear, constructors, main
- newlib integration with syscall stubs, and a heap that malloc can actually use
- printf over UART and over semihosting, selectable at build time
- A HAL layer over every peripheral, with a consistent API shape
- make run BOARD=<board> PROG=<x>.elf working identically on Verilator and KC705
- An onboarding path tested end to end on a contributor who has not seen the repo

LITERATURE REVIEW & THE NOTE IT PRODUCES

- Spec §27 (the software stack) and §27.1, §27.2 (semihosting)
- The newlib porting documentation and one existing RISC-V BSP for comparison
- Deliverable: a 2-page note specifying the HAL API shape before you write it, so the peripheral driver projects (M-01 to M-04) can target it

DELIVERABLES

- sw/bsp/ with crt0, syscalls, HAL and the linker script integration
- The make run flow and its documentation
- The onboarding test result, with the measured clone-to-running-C time

DEFINITION OF DONE

- make run BOARD=verilator PROG=hello.elf works from a clean clone
- printf works over both UART and semihosting
- A contributor new to the repo reaches running C in under a day, following docs alone

MILESTONES

W1–2	Reading and the HAL API note
W3–4	crt0 and the linker integration
W5–6	newlib, syscalls, malloc
W7–8	HAL and printf paths
W9	make run across boards
W10	Onboarding test and revision

Start here: Spec §27, NFR-9, EXECUTION_PLAN WP14 and §11

NOTES · QUESTIONS FOR YOUR MENTOR

RISCOF, Architectural Tests and Co-simulation in CI

Verification

DURATION	PRIORITY	PREREQUISITES
8 weeks	**P0**	Python, CI systems, patience with toolchains

Own the gate. RISCOF against Sail on every merge, wired into CI so that a second-year student learns within twenty minutes whether their change broke the ISA. This has to be green before the RTL exists — a verification harness built after the core is a harness that never gets built.

WHAT YOU WILL BUILD

- RISCOF with Sail as the golden model, running green on an empty core in Phase 0
- The full arch-test suite wired into per-PR CI within the 20-minute budget (NFR-3)
- Test selection per configuration, since the four configs implement different subsets
- Result reporting into the evidence-bundle format
- The nightly full-suite job, separate from the fast per-PR job

LITERATURE REVIEW & THE NOTE IT PRODUCES

- RISCOF documentation; the Sail RISC-V model and how to build it
- Spec §28 (verification architecture), particularly layers 2 and 3, and §28.5 (CI policy)
- Deliverable: a 1-page plan for keeping the per-PR job under 20 minutes as the suite grows, agreed before you build

DELIVERABLES

- verif/riscof/ with plugins, configuration and the CI job definitions
- The evidence-bundle report generator
- Coordination with M-11, who triages the failures your harness finds

DEFINITION OF DONE

- CI green on an empty core before Phase 1 starts — the Phase-0 exit criterion
- Per-PR job completes within 20 minutes; nightly job runs the full suite
- Nothing can be merged with the gate red

MILESTONES

W1	Reading and the CI budget plan
W2–3	RISCOF and Sail running locally
W4–5	CI integration and the runner
W6	Per-config test selection
W7–8	Reporting, nightly job, documentation

Start here: Spec §28, §28.5, §28.6, NFR-3, EXECUTION_PLAN WP12 and §10 week 3

NOTES · QUESTIONS FOR YOUR MENTOR

KC705 Board Port — DDR3 Bring-Up and Timing Closure

FPGA & Tools

DURATION	PRIORITY	PREREQUISITES
12 weeks	**P1**	Vivado, FPGA flow, timing constraints; M-12's memo in hand

Take MEDS-S1 from simulation to silicon. A board port is exactly four files by design, but making those four files correct means clocking, pin constraints, DDR3 calibration and timing closure at the declared frequency. Phase 3 is what unlocks the rest of the lab, and this project is most of Phase 3.

WHAT YOU WILL BUILD

- The four-file board port: board.yaml, board.xdc, board_top.sv, openocd.cfg
- Clock and reset topology, PLL configuration, and the CDC into the peripheral domain
- DDR3 via MIG, using M-12's evaluated configuration, through to calibration success
- The bring-up sequence in spec §29.2, executed in order, with each step recorded
- Timing closure at 50 MHz minimum (NFR-1), with the report published
- An out-of-context synthesis flow so accelerator work does not rebuild the whole SoC

LITERATURE REVIEW & THE NOTE IT PRODUCES

- Spec §29 (FPGA implementation) and §30 (area, timing and power budgets)
- M-12's Xilinx IP decision memo; the KC705 board user guide
- Deliverable: a bring-up log kept from day one, recording every step of §29.2 and every failure — this becomes the guide for the next board

DELIVERABLES

- fpga/boards/kc705/ with the four files and the build scripts
- The bring-up log and the published timing and utilisation reports
- The out-of-context synthesis flow and its documentation

DEFINITION OF DONE

- Every step of the §29.2 bring-up order passed, in order, and recorded
- Timing met at the declared frequency; utilisation report published
- A C program prints over UART from the FPGA, not from simulation

MILESTONES

W1–2	Reading, M-12 handover, constraint authoring
W3–4	Bitstream, clocks, LED, UART bring-up
W5–7	DDR3 and MIG calibration
W8–10	Full SoC integration and timing closure
W11–12	Out-of-context flow, reports, documentation

Start here: Spec §29, §30, NFR-1, NFR-8, EXECUTION_PLAN WP16

NOTES · QUESTIONS FOR YOUR MENTOR

Completion Buffer, Retire Logic and the MXIF Port

Core

DURATION	PRIORITY	PREREQUISITES
12 weeks	**P0 — critical path**	Strong microarchitecture; formal or assertion experience valued

The hardest package in the project, and it sits on the critical path. The completion buffer is what allows in-order issue with out-of-order completion and in-order retire — which is what makes accelerator attachment possible without stalling the whole pipeline. MXIF, the tightly-coupled extension port, hangs off it. Assign the strongest person here and start early.

WHAT YOU WILL BUILD

- An 8-entry completion buffer with the entry format of spec §9.1
- Retire rules per §9.2, preserving precise exceptions including for offloaded instructions
- The MXIF port implementing MXIF-1.0: non-speculative issue, two-phase completion
- Offload at the retire pointer, so no coprocessor side effect is ever speculative
- SVA liveness properties per §9.4, running from day one rather than added at the end
- A conformance testbench driving every MXIF handshake case, including rejection and faults

LITERATURE REVIEW & THE NOTE IT PRODUCES

- Spec §9 in full, especially §9.3 (why eight entries) and §9.4 (the deadlock argument)
- INTERFACES.md §1 in full — MXIF-1.0 is normative and you may not amend it locally
- The OpenHW CV-X-IF specification, and our profile's deviations from it
- Deliverable: a written deadlock argument for your implementation, independent of the spec's, presented at a design review before RTL

DELIVERABLES

- rtl/core/completion/ with the buffer, retire logic and the MXIF port
- verif/conformance/tb_mxif_conformance.sv — a reusable asset that outlives every coprocessor
- The SVA property set and the deadlock argument

DEFINITION OF DONE

- Liveness properties pass under bounded model checking
- Conformance testbench covers every handshake case in INTERFACES.md §1
- MEDS-V attaches over this port with zero changes to core RTL — the real test

MILESTONES

W1–3	Reading, deadlock argument, design review
W4–6	Completion buffer and retire
W7–9	MXIF port and two-phase completion
W10–11	Conformance testbench and SVA
W12	MEDS-V attachment trial with the MEDS-V team

Start here: Spec §9, §19; INTERFACES.md §1; EXECUTION_PLAN WP5, risk R8

NOTES · QUESTIONS FOR YOUR MENTOR

Load/Store Unit, Store Buffer, PMP and Atomics

Memory

DURATION	PRIORITY	PREREQUISITES
12 weeks	**P0**	Strong microarchitecture; memory-model literacy

Own memory as the core sees it. The LSU, the store buffer, PMP and PMA checking, and the A-extension atomics. This module decides what is legal, what is ordered and what traps — and in v1 it also stalls scalar memory while coprocessor memory is outstanding, a deliberate simplification whose cost you are expected to measure and report.

WHAT YOU WILL BUILD

- LSU with the full set of load and store widths, sign extension and alignment handling
- A store buffer with correct load-forwarding from pending stores
- 16 PMP regions with standard granularity and locking
- The PMA check unit — coordinate with M-05, who writes its testbench
- AMO and LR/SC per the A extension, including the reservation set rules
- The scalar/coprocessor memory interlock of INTERFACES.md §1.5, and its measured cost

LITERATURE REVIEW & THE NOTE IT PRODUCES

- Spec §14 (load/store unit) and §11 (PMP and PMA)
- The RISC-V memory model chapter, and the A-extension specification
- Deliverable: a 2-page note on the interlock — what it costs, how you will measure it, and what a v2 disambiguation scheme would have to beat

DELIVERABLES

- rtl/core/lsu/ with the LSU, store buffer, PMP and PMA units
- Unit testbenches, plus support for M-05's PMA testbench
- The interlock cost measurement, published

DEFINITION OF DONE

- A-extension architectural tests pass against Sail
- PMP tests pass; M-05's PMA coverage closes against your implementation
- The interlock cost is measured on the benchmark suite and reported honestly

MILESTONES

W1–2	Reading, interlock note, design review
W3–5	LSU and store buffer
W6–7	PMP and PMA
W8–10	Atomics and reservations
W11–12	Interlock measurement, arch-tests, documentation

Start here: Spec §11, §14; INTERFACES.md §1.5; SCOPE_CONTRACT §3; EXECUTION_PLAN WP6

NOTES · QUESTIONS FOR YOUR MENTOR

Instruction and Data Caches, Zicbom and the SRAM Wrapper

Memory

DURATION	PRIORITY	PREREQUISITES
12 weeks	**P1**	Cache microarchitecture; R-02 landing first is strongly preferred

Build the caches and — just as importantly — the SRAM wrapper that keeps this design ASIC-clean. Blocking, 2-way, write-back with a write buffer: correct and adequate rather than clever, because non-blocking is a measurable v2 optimisation and a better project once there is a baseline to beat.

WHAT YOU WILL BUILD

- Parameterised I$ and D$: 2-way, 64-byte lines, configurable size
- Write-back D$ with a write buffer, and the coherence-free software model via Zicbom
- cbo.clean, cbo.flush and cbo.inval, with correct interaction with the store buffer
- meds_sram_wrapper — every memory in the design goes through it, no exceptions (NFR-5)
- Cache testbenches: hit, miss, eviction, write-back ordering, cbo semantics
- Measured hit rates on the benchmark suite, published as the v1 baseline

LITERATURE REVIEW & THE NOTE IT PRODUCES

- Spec §15 (caches) and §17 (the SRAM wrapper)
- The Zicbom specification; SCOPE_CONTRACT §3 on why coherence is deferred
- Deliverable: a 2-page note on the Zicbom software contract — what software must do around a DMA buffer — which becomes the driver template's documentation

DELIVERABLES

- rtl/core/cache/ and rtl/common/meds_sram_wrapper.sv with unit testbenches
- The Zicbom software contract note, handed to the BSP team
- Baseline hit-rate measurements across the workload suite

DEFINITION OF DONE

- No memory anywhere in the design is instantiated outside meds_sram_wrapper
- cbo semantics proven correct against a DMA-style testbench
- Hit rates published, giving a v2 cache project a baseline

MILESTONES

W1–2	Reading, Zicbom contract note, design review
W3–4	SRAM wrapper and the memory abstraction
W5–7	I$
W8–10	D$ and write buffer
W11–12	Zicbom, testbenches, baseline measurement

Start here: Spec §15, §17; NFR-5; SCOPE_CONTRACT §3; EXECUTION_PLAN WP7

NOTES · QUESTIONS FOR YOUR MENTOR

The SoC Generator — soc.yaml to RTL, Linker, Headers and DTS

FPGA & Tools

DURATION	PRIORITY	PREREQUISITES
12 weeks	**P0**	Strong Python; templating; SystemVerilog literacy

Build the single source of truth. One soc.yaml generates the SoC top level, the linker script, the C headers, the device tree, the memory-map documentation, the Verilator top, the OpenOCD config and the PMA decode logic. The failure this prevents — a memory map correct in the RTL and wrong in the device tree — is otherwise inevitable and brutal to debug.

WHAT YOU WILL BUILD

- The soc.yaml schema, versioned, with validation and clear error messages
- Generators for: SoC top-level RTL, link.ld, C headers, .dtsi, memory_map.md
- Generators for the Verilator top, the OpenOCD configuration and the PMA decode logic
- Golden tests: a known soc.yaml produces byte-identical outputs, checked in CI
- The peripheral-addition path documented well enough that a mentee can use it unaided

LITERATURE REVIEW & THE NOTE IT PRODUCES

- Spec §26 (the SoC generator) and Appendix B (default memory map)
- One existing SoC generator — Chipyard, LiteX or similar — and what it gets right and wrong
- Deliverable: a 2-page schema design note, reviewed before implementation, because the schema is an interface and changing it later is expensive

DELIVERABLES

- generator/ with the schema, generators, templates and golden tests
- The schema note and a contributor guide for adding a peripheral
- Generated outputs wired into the build so no hand-maintained duplicates remain

DEFINITION OF DONE

- Every artefact in spec §26 is generated; no hand-maintained duplicate exists anywhere
- Golden tests pass in CI
- A contributor who has only done rv-workshop adds a peripheral and sees it from C, in under a day, following the documentation alone (NFR-9)

MILESTONES

W1–2	Reading, schema design note, review
W3–5	RTL and linker-script generation
W6–7	Headers, DTS, documentation generation
W8–9	Verilator top, OpenOCD, PMA decode
W10–11	Golden tests and CI
W12	Contributor guide and the NFR-9 test

Start here: Spec §26, Appendix B; NFR-9; EXECUTION_PLAN WP10

NOTES · QUESTIONS FOR YOUR MENTOR

RVFI Trace Port and the Spike Co-simulation Harness

Verification

DURATION	PRIORITY	PREREQUISITES
10 weeks	**P0**	SystemVerilog and C++; DPI; debugging stamina

The highest-value verification investment in the project. A directed test costs twenty minutes and covers one case; this harness costs three days and covers every case any program ever exercises, forever. Without it the only signal is "the answer is wrong"; with it the signal names the instruction, the PC, the register and the expected value.

WHAT YOU WILL BUILD

- An RVFI port driven from the first pipeline commit, per INTERFACES.md §5
- The MEDS rvfi_v_* extension group for vector state, including byte-granular write masks
- A Spike co-simulation harness comparing every retired instruction over DPI
- Divergence reporting that names instruction number, PC, mnemonic, and expected versus actual
- CI integration on every PR, within the NFR-3 budget
- Trace capture on failure, so a divergence can be reproduced offline

LITERATURE REVIEW & THE NOTE IT PRODUCES

- The RVFI specification from riscv-formal; INTERFACES.md §5
- Spec §28.1 and §28.2 — why co-simulation dominates, and the RVFI contract
- The MEDS-V book's co-simulation chapter, which solved this problem once already
- Deliverable: a 2-page note on how vector state is checked, since comparing final register contents is exactly what misses undisturbed-tail bugs

DELIVERABLES

- rtl/core/rvfi/ and verif/cosim/ with the harness and the DPI layer
- CI job running co-simulation on every PR
- The vector-checking note, and documentation on reading a divergence report

DEFINITION OF DONE

- Every retired instruction is compared against Spike across the whole benchmark suite
- A deliberately injected bug is caught and correctly localised by the report
- Runs on every PR within the CI budget

MILESTONES

W1–2	Reading, vector-checking note, design review
W3–4	RVFI port
W5–7	Spike harness and the DPI layer
W8	Divergence reporting
W9–10	CI integration, fault injection testing, documentation

Start here: Spec §28.1, §28.2; INTERFACES.md §5; EXECUTION_PLAN WP11

NOTES · QUESTIONS FOR YOUR MENTOR

Debug Module, JTAG DTM, OpenOCD and Semihosting

FPGA & Tools

DURATION	PRIORITY	PREREQUISITES
10 weeks	**P0**	JTAG, GDB internals, C; T-03's privilege FSM in place

Deliver the capability that unlocks the whole lab. Until an arbitrary ELF can be loaded and debugged over JTAG with no resynthesis, every software change costs a bitstream build and the platform is useful only to the people building it. This is the Phase-3 exit criterion, and everything else in Phase 3 is downstream of it.

WHAT YOU WILL BUILD

- RISC-V Debug Module integration using pulp-platform/riscv-dbg, plus the JTAG DTM
- Halt, resume, single step, and two instruction-address triggers
- Abstract commands for register access, and system-bus access for memory
- OpenOCD configuration generated from soc.yaml, and a working GDB connection
- Semihosting: open, read, write, close and lseek proxied to the host filesystem
- Debug entry and exit correctness against the privilege FSM, proven by testbench

LITERATURE REVIEW & THE NOTE IT PRODUCES

- The RISC-V External Debug Support specification
- Spec §13 (debug) and §27.2 (semihosting)
- Deliverable: a 2-page note on debug-mode entry and its interaction with traps and the completion buffer, agreed with the R-01 and T-03 owners before RTL

DELIVERABLES

- rtl/debug/ with the DM and DTM integration and its testbench
- Generated OpenOCD configuration and the GDB workflow documentation
- Semihosting implementation in the BSP, with the host-side proxy

DEFINITION OF DONE

- openocd and gdb load and debug an arbitrary ELF over JTAG with no resynthesis — the Phase-3 exit criterion
- Semihosting moves a multi-megabyte file to the target without an SD card
- Halt, step, breakpoint and register access all proven on hardware

MILESTONES

W1–2	Reading, debug-entry note, design review
W3–5	DM and DTM integration
W6–7	OpenOCD and GDB bring-up
W8–9	Semihosting
W10	Hardware validation and documentation

Start here: Spec §13, §27.2; RISC-V Debug Spec; EXECUTION_PLAN WP15, risk R6

NOTES · QUESTIONS FOR YOUR MENTOR

Accelerator Socket and the Conformance Testbenches

Memory

DURATION	PRIORITY	PREREQUISITES
10 weeks	**P1**	AXI4, CDC discipline, interface design judgement

Build the loosely-coupled attachment point most ML thesis projects will use, and the conformance testbench that keeps it honest. Clock-domain crossing lives in the socket so accelerator authors never write synchronisers. This is the module that turns MEDS-S1 from a processor into a platform.

WHAT YOU WILL BUILD

- meds_s1_accel_socket with AXI4-Lite config window, AXI4 DMA master and CDC, per spec §20.1
- The mandatory register map of §20.2 — including PERF_CYCLES and PERF_STALLS at fixed offsets
- Level-sensitive IRQ synchronisation into the PLIC
- ASYNC=0/1 support so an accelerator can run in its own clock domain safely
- verif/conformance/tb_socket_conformance.sv covering the register map, IRQ assert and clear, abort, error reporting, and DMA to both legal and illegal regions
- A driver template with the cache maintenance already correct, since that is what students miss

LITERATURE REVIEW & THE NOTE IT PRODUCES

- Spec §20 in full, §21 (choosing a coupling mechanism), §22 (data movement patterns)
- INTERFACES.md §4 — the socket interface is normative
- Deliverable: a 2-page note on the CDC strategy and why accelerator authors are forbidden from writing their own synchronisers (NFR-6)

DELIVERABLES

- rtl/socket/ with the socket RTL and its parameterisation
- The socket conformance testbench — a reusable asset that outlives every accelerator
- The driver template and the accelerator author's guide

DEFINITION OF DONE

- The conformance testbench covers every case in INTERFACES.md §4
- Two trivial example accelerators attach and pass conformance
- An accelerator author following the guide alone attaches a stub in under a day

MILESTONES

W1–2	Reading, CDC note, design review
W3–5	Socket RTL, register window, DMA path
W6–7	CDC and IRQ synchronisation
W8–9	Conformance testbench
W10	Driver template and the author's guide

Start here: Spec §20, §21, §22; INTERFACES.md §4; NFR-6; EXECUTION_PLAN WP17, risk R10

NOTES · QUESTIONS FOR YOUR MENTOR